

Technical Report: Fireworks Agentic RAG for 10-K Analysis

This is a brief report on what was built, how the system is structured, how retrieval works from SQL and PDFs, how the system is evaluated, key trade-offs, and planned improvements.

1. What I Built

I built a local agentic RAG system that helps financial analysts answer research questions over Apple, Microsoft, and Alphabet using both structured financial data from a SQLite database (`financials.db`) and narrative content from six 10-K filings (FY2024–FY2025). An LLM-based agent decides whether to retrieve information from SQL, PDFs, or both, and then synthesizes a grounded final answer. The system runs locally except for Fireworks API calls used for routing, text-to-SQL generation, embeddings, synthesis, and evaluation.

The project includes a FastAPI backend (`http://localhost:8000/api/chat`) and a browser-based chat UI (`http://localhost:8000`) with markdown answers, sample questions, SQL evidence, and PDF chunk references. Supporting scripts handle PDF indexing (`build_pdf_index.py`), answer generation (`generate_answers.py`), and evaluation (`evaluate.py`). Final outputs are stored in `questions/dev_answers.json` and `eval_results.json`.

2. How the System Is Structured

The pipeline follows a plan-retrieve-synthesize pattern:

User Question -> Router (LLM decides SQL, PDF, or Both) -> SQL Tool / PDF Tool -> Synthesizer (LLM) -> Grounded Answer

Core Components

- **Agent (`agent.py`):** Plans tool calls via LLM routing, executes retrieval, and synthesizes answers. Includes heuristic section routing, query expansion, and keyword reranking.
- **SQL Tool (`sql_tool.py`):** Text-to-SQL against `financials.db` using `deepseek-v4-pro`, with preset queries for patterns where LLM-generated SQL was unreliable.
- **PDF Tool (`pdf_tool.py`):** Section-aware chunking, `nomic-embed-text-v1.5` embeddings, ChromaDB storage, and metadata-filtered retrieval.

3. How I Retrieve from SQL and PDFs

SQL Retrieval

SQL retrieval handles questions that need exact numbers: revenue, percentages, growth rates, balance-sheet ratios, segment totals, and cross-company comparisons. The LLM receives a schema description covering companies, income_statements, balance_sheets, segment_revenue, and geographic_revenue. All full-year rows use period_type = 'FY'. The generated SQL is executed and results are formatted as readable tables for synthesis.

PDF Retrieval

PDF retrieval handles narrative and explanatory questions: risk factors, segment definitions, management discussion, and strategic commentary. PDFs are parsed with pdfplumber. Narrative text is chunked at 1200 characters with 300-character overlap. Chunk boundaries follow SEC Item headers detected line-by-line (Item 1A Risk Factors, Item 7 MD&A, Item 8 Financial Statements, and others) so each chunk carries correct section metadata. Tables are extracted separately, converted to pipe-delimited text, and chunked at 900 characters with no overlap, tagged as table chunks. All chunks are embedded and stored in ChromaDB with metadata for filtering at query time.

Routing Rules

- Numbers -> SQL
- Narrative / qualitative -> PDF
- Numbers + explanation -> Both tools
- Multi-step comparisons -> chain multiple tool calls

4. How I Evaluate the System

The assignment provides ten public dev questions and a gold answer key (dev_questions_with_answers.json) but no official scoring harness. I built scripts/evaluate.py to measure correctness systematically. First, generate_answers.py runs each dev question through the full agent and writes questions/dev_answers.json. Then evaluate.py compares each predicted answer to the gold answer using a method suited to the question type.

For numeric questions (for example, Apple FY2025 revenue or Microsoft US revenue percentage), I use fuzzy_numeric scoring: extract the primary number from the predicted answer (handling billion, million, and trillion suffixes), then accept answers within plus or minus 2% of the gold value. For questions where naming the correct company is the main requirement (fastest growth, highest current ratio), I use entity_match: check whether the gold company name or ticker appears in the prediction. For open-ended synthesis questions, I use an LLM judge on Fireworks that scores factual correctness and completeness with partial credit, combined with a deterministic list heuristic that counts how many gold bullet points appear in list style answers. The final score per question is the maximum of the judge score and the heuristic where both apply.

On the latest run, the system scored 92.5% overall (9.25 out of 10). Seven questions scored 1.0, and three scored 0.75: q_006 (supply chain risks missing some gold themes), q_017 (YouTube revenue change not retrieved from MD&A tables), and q_025 (synthesis leaked chain-of-thought

instead of a concise final answer). These gaps inform the improvement priorities below. Results are saved to eval_results.json for reproducibility.

Question	Tier	Score	Method
q_001 Apple FY2025 revenue	1	1.00	fuzzy_numeric
q_006 Apple supply chain risks	1	0.75	llm_judge
q_008 Microsoft US revenue %	1	1.00	fuzzy_numeric
q_011 Fastest revenue growth	2	1.00	entity_match
q_012 Greater China + why	2	1.00	llm_judge
q_014 MSFT segments + revenue	2	1.00	llm_judge
q_017 Google Services / YouTube	2	0.75	llm_judge
q_018 Highest current ratio	2	1.00	entity_match
q_022 Largest revenue increase	3	1.00	llm_judge
q_025 Services vs iPhone growth	3	0.75	llm_judge

Overall score: 92.5% (9.25 / 10)

5. Trade-offs

I chose agentic routing over always retrieving from both SQL and PDF because it reduces latency, API cost, and noise from irrelevant chunks, at the cost of depending on router accuracy. I use a single LLM (deepseek-v4-pro) for routing, SQL generation, and synthesis to keep the stack simple rather than tuning separate models per task.

For SQL, I default to text-to-SQL for flexibility but added preset queries only where evaluation proved the LLM was unreliable. That trades some generality for much higher accuracy on Tier 2 and Tier 3 numeric questions. For PDFs, I use local ChromaDB with cosine similarity rather than a managed vector service so reviewers can run everything offline after indexing. Narrative chunks are relatively large (1200 characters with 300 overlap) to preserve context in long 10-K sections; section-aware splitting reduces the risk of mixing unrelated items in one chunk.

Retrieval uses top-8 with keyword reranking rather than a cross-encoder reranker. Reranking is fast and improved supply-chain retrieval without adding another model, but it requires hand-tuned term lists. The chat UI is a single HTML file with no frontend build step so reviewers can use it immediately. I accept that prose answers are judged partly by LLM, which is flexible but less deterministic than numeric scoring.

Decision	Choice	Rationale
----------	--------	-----------

LLM	deepseek-v4-pro	Strong financial reasoning; single model simplifies ops
Embeddings	nomic-embed-text-v1.5	Good prose search; available on Fireworks
Vector store	ChromaDB (local)	No external service; rebuildable from PDFs
Chunk size	800 / 150 overlap	Balances context and precision
Top-K	8, rerank to 8	Recall for lists; reranking trims noise
SQL	LLM + presets	Presets only where eval showed SQL failures
UI	Single-page HTML	Zero build step for reviewers

6. What I Would Improve With More Time

The biggest improvement would be adding stronger safeguards during answer synthesis for Tier 3 questions. Responses like q_025 should never expose internal reasoning phrases such as “We are asked” or “Let’s calculate.” This can be fixed with a post-processing validation step and a stricter retry prompt to ensure only concise final answers are returned.

For questions like q_017, I would improve PDF retrieval by adding a second retrieval pass focused specifically on MD&A and Financial Statement table sections with fiscal-year filters. This would help retrieve year-over-year revenue values such as YouTube advertising revenue more reliably. For q_025, I would also use a SQL-first hybrid pipeline where Services and iPhone growth rates are computed directly from structured SQL data, while PDFs are used only for qualitative business commentary.

Additional improvements would include replacing keyword reranking with a stronger cross-encoder reranker, adding an SQL self-correction loop that retries failed queries using SQLite error feedback, supporting inline citations with page numbers, enabling SSE streaming for intermediate reasoning steps, and building a private regression benchmark to avoid overfitting to the ten public development questions. Although the current system achieves a 92.5% public dev score, the hidden evaluation set will likely require stronger retrieval accuracy and tighter synthesis control.

AI Tools Used

Claude (Anthropic) and Cursor were used for scaffolding, debugging routing and retrieval issues, UI work, and report drafting. All routing, SQL generation, synthesis, embeddings, and evaluation judging run on Fireworks-hosted model as required by the assignment.